Hawkin Dynamics Power Query Scripts

Welcome to the official Hawkin Dynamics Power Query repository. This collection of scripts and templates enables you to pull athlete performance data directly from the Hawkin Dynamics Cloud API into Microsoft Power BI using the Power Query (M) formula language.

Whether you are building a simple weekly report or a complex historical analysis dashboard, these tools bridge the gap between your raw force plate data and actionable insights.

Repository Structure

The repository is organized into three directories, each representing a different approach to connecting with the API. Choose the one that best fits your workflow:

	Generics	Parameterized Template	Template Files
Audience	Developers, debugging	Power users building custom reports	Anyone who wants a ready-to-go dashboard
Setup effort	Manual edits per script	One-time parameter + function setup	Open .pbit and enter credentials
Maintenance	Edit each script individually	Update one parameter, all queries inherit	Re-download latest template
Schema handling	Dynamic (automatic)	Dynamic (automatic)	Dynamic (automatic)
Date range strategy	Single API call	Automatic 30-day chunking	Automatic 30-day chunking

1. Template Files (Recommended for Beginners)

Location: `/Template Files`

The fastest way to get started. A pre-built Power BI Template (`.pbit`) file contains all queries, functions, and table relationships pre-configured.

- **Contains:**
  - `hd-api-template-v1.20260323.pbit` — the Power BI template
  - `data-queries/` — source code for each data table query
  - `functions/` — reusable Power Query functions used by the data queries
  - `release-notes-*.md` — version history
- **Why use this?**
  - **Zero coding** — you do not need to write or edit any Power Query code.
  - **Instant structure** — tables for Athletes, Teams, Groups, Tags, Test Types, and all 11 test types are pre-linked in the data model.

- **Lightweight** — the `.pbit` file contains the structure but no data, making it easy to share.

- **How to use:**

  1. Download `hd-api-template-v1.20260323.pbit` from the `Template Files/` directory.
  2. Double-click the file to open it in Power BI Desktop.
  3. A pop-up window will appear asking for your parameters:
     - **SecretKey** — paste your Integration Key.
     - **reg** — your region: `Americas`, `Europe`, or `Asia/Pacific`.
     - **orgName** — your Organization ID (leave blank if not applicable).
     - **Organization Start Date** — the earliest date from which to pull test data.
  4. Click **Load**. Power BI will connect to the API, build the data model, and populate all tables.

- **How it works under the hood:**

  - The template uses Power BI Parameters to store your credentials and region.
  - An `Auth` query calls `funcAuth()` to obtain an access token, which is cached and reused across all data queries.
  - The `epochNow()` function checks token expiration — if the token has expired, `funcAuth()` is called again automatically.
  - Each test-type query (e.g., CMJ, SquatJump) breaks the full date range into 30-day chunks and calls `funcTests()` for each chunk, then combines the results.
  - `funcTests()` uses dynamic schema handling to automatically discover and expand all metric columns returned by the API.

---

2. Parameterized Template (Best for Power Users)

Location: `/Parameterized Template`

These scripts use the same modular, function-based architecture as the Template Files but are designed for users who want to build their own Power BI report from scratch. Instead of opening a pre-built `.pbit`, you import the functions and data queries individually into your own Power BI project.

- **Contains:**

  - `data-queries/` — one `.pq` file per data table (Athletes, Teams, Groups, Tags, TestTypes, ForceTime, and 11 test-type queries)
  - `functions/` — reusable Power Query functions:
    - `funcAuth.pq` — authenticates with the API and returns an access token
    - `funcTests.pq` — fetches test data for any test type with dynamic schema handling
    - `funcAthletes.pq` — fetches and expands athlete roster data
    - `funcTeams.pq` — fetches team data
    - `funcTags.pq` — fetches tag data
    - `epochNow.pq` — returns current UTC time as epoch seconds for token validation
  - `hd-api-template-v1.20260323.pbit` — a copy of the same template from Template Files

- **Why use this?**

  - **Full control** — build your own report layout, choose which queries to include, and customize the data model.
  - **Single-point credential management** — define your Integration Key, region, and org once as Power BI Parameters; every query inherits them.
  - **Automatic date chunking** — test-type queries break large date ranges into 30-day API calls, avoiding timeouts on large datasets.
  - **Dynamic schema** — new metrics added to the API appear automatically without script updates.

- **Video walkthrough:** Watch the setup tutorial on Loom

- **How to set up in Power BI:**

  1. **Create Parameters.** In Power BI Desktop, go to **Home** > **Transform Data** > **Manage Parameters**. Create these parameters (names must match exactly):

     - `SecretKey` (Text) — your Integration Key
     - `reg` (Text) — your region: `Americas`, `Europe`, or `Asia/Pacific`
     - `orgName` (Text) — your Organization ID (leave blank if not applicable)
     - `Organization Start Date` (Date) — the earliest date for test data

  2. **Import the functions first.** For each file in the `functions/` folder:

     - In Power BI, go to **Home** > **Get Data** > **Blank Query**.
     - Open the **Advanced Editor**, paste the contents of the `.pq` file, and click **Done**.
     - Rename the query to match the file name exactly (e.g., `funcAuth`, `funcTests`, `epochNow`). This is critical — data queries reference functions by name.

  3. **Import the Auth query.** Import `data-queries/Auth.pq` as a query named `Auth`. This query calls `funcAuth()` with your parameters and caches the access token.

  4. **Import data queries.** For each data table you want (e.g., `Athletes.pq`, `CMJ.pq`, `Teams.pq`):

     - Create a new blank query, paste the code from the `.pq` file, and rename it appropriately.
     - The query will automatically reference `Auth` for the access token and call the appropriate function.

  5. **Click Close & Apply** to load data into your report.

- **Architecture diagram:**

```
Parameters (SecretKey, reg, orgName, Organization Start Date)
     |
     v
Auth query --> funcAuth() --> access token (cached)
     |
     v
Data queries (CMJ, SquatJump, Athletes, etc.)
     |              |
     v              v
funcTests()    funcAthletes() / funcTeams() / funcTags()
     |
```

```
      v
Dynamic schema expansion --> typed & reordered output table
```

---

3. Generics (Standalone Scripts)

Location: `/Generics`

Standalone, self-contained scripts where all configuration — API key, region, test type, date filters — is defined directly at the top of each file. Each script handles its own authentication and data retrieval independently.

- **Contains:**
  - `Athletes.pq` — athlete roster
  - `Teams.pq` — team list
  - `Groups.pq` — group list
  - `Tags.pq` — tag list
  - `TestTypes.pq` — available test types
  - `ForceTime.pq` — force-time curve data for a single test
  - `Tests by Test Type/` — one script per test type (CmJump, SquatJump, DropJump, Isometric, etc.)
- **Why use this?**
  - **Self-contained** — each script works independently; copy one file, paste it into Power BI, and it runs.
  - **Transparent** — all logic is visible in a single linear flow, making it ideal for learning how the API works or debugging connection issues.
  - **No dependencies** — no need to set up parameters, functions, or an Auth query.
- **How to use:**
  1. Open the desired `.pq` file in a text editor.
  2. Locate the **User Configuration Section** at the top:

     ```
     SecretKey = "YOUR_INTEGRATION_KEY",
     orgName = null,
     reg = "Americas",
     startDate = "",
     endDate = "",
     ```

  3. Replace the placeholder values with your actual credentials and desired filters.
  4. Copy the entire script and paste it into Power BI's **Advanced Editor** (Home > Get Data > Blank Query > Advanced Editor).
  5. Click **Done** to execute.
- **Limitations compared to Parameterized Template:**

- Each script authenticates independently (no token caching/reuse).
- Credentials must be updated in every script individually.
- Test data is fetched in a single API call (no 30-day chunking), which may time out for very large date ranges.

Comparing the Two Power BI Setup Approaches

Generics: One Script = One Query

With the Generics approach, each `.pq` file is a complete, standalone query. You paste it into Power BI and it handles everything internally: authentication, API call, data expansion, and typing. This means:

- **Setup:** Paste one script per query into the Advanced Editor. Edit the config variables at the top of each one.
- **In Power BI:** Each query appears as an independent item with no shared dependencies. The "Applied Steps" pane shows every step from authentication through final output.
- **Trade-off:** Simple to understand, but credentials are duplicated across queries and there is no token reuse.

Parameterized Template: Functions + Parameters

The Parameterized Template approach separates concerns into reusable functions and lightweight data queries. This means:

- **Setup:** First, create Power BI Parameters and import the shared function queries. Then import the data queries, which are short scripts that call into the functions.
- **In Power BI:** You will see two types of queries in the Queries pane:
  - **Functions** (funcAuth, funcTests, etc.) — shown with a function icon. These are not data tables; they are reusable logic.
  - **Data queries** (CMJ, Athletes, etc.) — these call the functions and produce the actual tables.
  - **Parameters** (SecretKey, reg, orgName, Organization Start Date) — shown with a parameter icon. Change a value here and all queries update.
- **Trade-off:** Requires more initial setup, but is far easier to maintain, avoids credential duplication, handles large datasets via chunking, and automatically adapts to API schema changes.

Supported Test Types

Test Type	Generic Script	Parameterized/Template Query	Test Type ID
Countermovement Jump	CmJump.pq	CMJ.pq	`7nNduHeM5zETPjHxvm7s`
CMJ Rebound	CmJumpRebound.pq	CMJ_Rebound.pq	`gyBETpRXpdr63Ab2E0V8`
Squat Jump	SquatJump.pq	SquatJump.pq	`QSy5LM7CZQJ1MZE37lMO`
Drop Jump	DropJump.pq	DropJump.pq	`5pRSUQVSJVnxijpPMck3`

Test Type	Generic Script	Parameterized/Template Query	Test Type ID
Drop Landing	DropLanding.pq	DropLanding.pq	`umnEZPgi6gG9QwMvGSig`
Isometric	Isometric.pq	Isometric.pq	`ubeWMPN1lJFbuQbEqc1C`
Multi Rebound	MultiReb.pq	MultiReb.pq	`r4fhrkPdYlLxYQxEeM78`
Free Run	FreeRun.pq	FreeRun.pq	`pqgf2TPUOQOQs6r0HQWb`
Weigh In	WeighIn.pq	WeighIn.pq	`uSvrZaHqDvfZn7S5KnHt`
TS Free	TS_Free.pq	TS_Free.pq	`4KlQgKmBxbOY6uKTLDFL`
TS Isometric	TS_Isometric.pq	TS_Isometric.pq	`wGBM3RRHTBVT8GhAOi0Z`

Dynamic Schema Handling

All test-type scripts (across both Generics and Parameterized Template) use dynamic schema handling. Instead of listing specific metric column names, the scripts automatically discover all fields present in the API response:

- New metrics added to the API appear as columns automatically — no script updates needed.
- Metric columns are auto-typed as `number`.
- Columns are reordered: metadata first (id, timestamp, athlete info), then metrics alphabetically, then summary fields (count, lastSyncTime, date).

Prerequisites

- **Microsoft Power BI Desktop** — latest monthly version recommended. Free download from the Microsoft Store.
- **Hawkin Dynamics Integration Key:**
  - This is *not* your login password.
  - To find it, log in to the Hawkin Cloud, go to **Settings** > **Integrations**, and copy your unique API Key.
  - *Security Note:* Treat this key like a password. Do not share it in public forums or commit it to public repositories.

Support & Troubleshooting

- **Column Formatting:** After loading data, check your column types. Power Query may default numeric columns to "Text". Fix this by selecting the column and changing the "Data Type" in the Transform tab.
- **Token Expiration:** If using the Parameterized Template approach, tokens are validated automatically via `epochNow()`. If you encounter auth errors, try refreshing all queries.
- **Large Date Ranges:** The Parameterized Template scripts chunk requests into 30-day windows. If you experience timeouts with Generic scripts on large date ranges, consider switching to the Parameterized Template approach.

- **API Documentation:** For technical details on rate limits, data points, and field definitions, refer to the API docs PDF in the repo.

Release Notes

- v1.0.20260323 — Dynamic schema handling, function consolidation, folder restructure
- v1.0.20260109 — Bug fixes for endpoint URLs, updated templates